LAPORAN PRAKTIKUM

PEMROGRAMAN MOBILE

Pertemuan 4: Mengambil Data dari REST API (HTTP, Async, dan FutureBuilder)

Nama	:	Muhamad Dika Ramadhan
NIM	:	20240801045
Program Studi	:	Teknik Informatika
Mata Kuliah	:	Pemrograman Mobile
Pertemuan	:	4

1. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Menjelaskan pemrograman asinkron di Dart: Future, async, dan await.
2. Mengambil data dari REST API memakai paket http dan menguraikan JSON.
3. Membuat class model dengan konstruktor fromJson.
4. Menampilkan data asinkron dengan FutureBuilder lengkap dengan status loading, error, dan data.
5. Menangani kegagalan jaringan dan menyediakan tombol coba lagi.

2. Landasan Teori

Asinkron

Mengambil data dari internet membutuhkan waktu. Agar tampilan aplikasi tidak membeku selama menunggu, Dart memakai Future, yaitu objek yang berisi hasil yang baru tersedia nanti. Kata kunci await menunggu hasil tanpa memblokir UI, dan hanya boleh dipakai di dalam fungsi yang ditandai async.

REST API dan JSON

Aplikasi mengirim permintaan HTTP (misalnya GET) ke sebuah URL dan menerima respons berformat JSON. Kode status 200 berarti sukses, 404 berarti tidak ditemukan, dan 500 berarti kesalahan server. API yang dipakai adalah JSONPlaceholder (jsonplaceholder.typicode.com), yaitu API publik gratis untuk latihan.

Konsep	Fungsi
http.get(uri)	Mengirim permintaan GET dan mengembalikan Future<Response>.
jsonDecode(body)	Mengubah teks JSON menjadi Map atau List Dart.
fromJson	Konstruktor factory yang mengubah Map menjadi objek model.
FutureBuilder	Widget yang membangun UI berdasarkan status sebuah Future.
snapshot	Berisi status (connectionState), data (data), dan error (error).

3. Alat dan Bahan

- Flutter SDK, editor, dan emulator dari pertemuan sebelumnya
- Proyek baru: flutter create praktikum_4
- Koneksi internet (wajib) untuk mengakses JSONPlaceholder
- Paket http (flutter pub add http)

4. Langkah dan Hasil Praktikum

Bagian A. Mengenal Future dan FutureBuilder

Fungsi ambilSalam() mensimulasikan pengambilan data yang butuh 2 detik, tanpa internet.

```
Future<String> ambilSalam() async {
  await Future.delayed(const Duration(seconds: 2));
  return 'Halo dari masa depan!';
}

class _SalamPageState extends State<SalamPage> {
  late Future<String> _future;

  @override
  void initState() {
    super.initState();
    _future = ambilSalam();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Future')),
      body: Center(
        child: FutureBuilder<String>(
          future: _future,
          builder: (context, snapshot) {
            if (snapshot.connectionState ==
                ConnectionState.waiting) {
              return const CircularProgressIndicator();
            }
```

```
            if (snapshot.hasError) {
              return Text('Error: ${snapshot.error}');
            }
            return Text(snapshot.data!,
                style: const TextStyle(fontSize: 24));
          },
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
Future<String>	Tipe data untuk nilai String yang baru tersedia di masa depan (setelah proses selesai).
async	Menandai fungsi sebagai asinkron; otomatis mengembalikan Future dan mengizinkan pemakaian await.
await Future.delayed(Duration(seconds: 2))	Menunggu 2 detik tanpa memblokir UI. Dipakai untuk mensimulasikan proses lambat seperti pengambilan data dari internet.
late Future<String> _future	Variabel yang diisi nanti (di initState), bukan saat dideklarasikan. Kata late menjamin nilainya sudah ada sebelum dipakai.
initState()	Dipanggil sekali saat State dibuat. Future dibuat di sini agar tidak diulang setiap build().
super.initState()	Memanggil initState milik kelas induk; wajib dipanggil lebih dulu.
FutureBuilder<String>	Widget yang memantau Future dan membangun UI sesuai statusnya (menunggu, error, atau selesai).
future: _future	Future yang dipantau oleh FutureBuilder.
builder: (context, snapshot)	Fungsi yang dipanggil setiap status Future berubah; snapshot menyimpan status, data, dan error.
ConnectionState.waiting	Status saat Future belum selesai; ditampilkan CircularProgressIndicator.
snapshot.hasError / snapshot.error	Memeriksa dan mengambil galat bila Future gagal.
snapshot.data!	Hasil Future setelah selesai; tanda ! menegaskan nilainya tidak null.

Catatan penting: Future dibuat di initState(), bukan di dalam build(). Jika dibuat di build(), Future baru akan dibuat setiap layar dibangun ulang sehingga proses pengambilan data berulang terus.

Bagian B. Menyiapkan Paket HTTP

```
flutter pub add http
```

Agar aplikasi release Android dapat mengakses internet, izin berikut ditambahkan di android/app/src/main/AndroidManifest.xml, tepat di atas tag <application:

```
<!-- android/app/src/main/AndroidManifest.xml,
     tepat di atas tag <application -->
<uses-permission android:name="android.permission.INTERNET"/>
```

Bagian	Penjelasan
flutter pub add http	Menambahkan paket http ke pubspec.yaml sekaligus mengunduhnya.
android.permission.INTERNET	Izin Android agar aplikasi diperbolehkan mengakses jaringan. Mode debug biasanya sudah memilikinya, tetapi build release membutuhkannya.

Bagian C. Model dan Fungsi Pengambil Data

Data dari endpoint /users berupa daftar objek JSON dengan kolom id, name, email, phone, dan website. Kode berikut ditambahkan di bawah import, bersama dua import baru di bagian paling atas.

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class Pengguna {
  final int id;
  final String name;
  final String email;
  final String phone;
  final String website;

  const Pengguna({
    required this.id,
    required this.name,
    required this.email,
    required this.phone,
    required this.website,
  });

  factory Pengguna.fromJson(Map<String, dynamic> json) {
    return Pengguna(
      id: json['id'] as int,
      name: json['name'] as String,
      email: json['email'] as String,
```

```
      phone: json['phone'] as String,
      website: json['website'] as String,
    );
  }
}

Future<List<Pengguna>> ambilPengguna() async {
  final uri = Uri.parse(
      'https://jsonplaceholder.typicode.com/users');
  final response =
      await http.get(uri).timeout(const Duration(seconds: 10));

  if (response.statusCode != 200) {
    throw Exception(
        'Gagal memuat data (kode ${response.statusCode})');
  }

  final List<dynamic> data = jsonDecode(response.body);
  return data
      .map((e) => Pengguna.fromJson(e as Map<String, dynamic>))
      .toList();
}
```

Bagian Kode	Penjelasan
import 'dart:convert'	Pustaka bawaan Dart yang menyediakan jsonDecode().
import 'package:http/http.dart' as http	Mengimpor paket http dengan awalan http agar pemanggilan jelas, misalnya http.get().
class Pengguna	Model data satu pengguna dengan properti id, name, email, phone, dan website.
final	Properti tidak dapat diubah setelah objek dibuat.
required	Parameter bernama wajib diisi saat membuat objek.
factory Pengguna.fromJson(Map<String, dynamic> json)	Konstruktor factory yang mengubah Map hasil jsonDecode menjadi objek Pengguna.
json['id'] as int	Mengambil nilai dari kunci "id" dan memastikan tipenya int (casting).
Future<List<Pengguna>> ambilPengguna() async	Fungsi asinkron yang mengembalikan daftar Pengguna.
Uri.parse('...')	Mengubah String URL menjadi objek Uri.
await http.get(uri)	Mengirim permintaan GET dan menunggu respons tanpa memblokir UI.
.timeout(Duration(seconds: 10))	Membatalkan permintaan jika tidak ada respons dalam 10 detik, sehingga aplikasi tidak menunggu selamanya.

Bagian Kode	Penjelasan
response.statusCode != 200	Memeriksa apakah server membalas dengan sukses.
throw Exception(...)	Melempar galat bila status bukan 200; galat ini nanti ditangkap FutureBuilder dan tersedia di snapshot.error.
jsonDecode(response.body)	Mengubah teks JSON pada body respons menjadi List<dynamic>.
.map(...).toList()	Mengubah setiap elemen JSON menjadi objek Pengguna lalu mengumpulkannya dalam List.

Bagian D. Menampilkan Data dengan FutureBuilder

Halaman PenggunaPage menangani tiga keadaan: memuat, galat (dengan tombol coba lagi), dan data. Properti home pada MyApp diubah menjadi const PenggunaPage().

```
class _PenggunaPageState extends State<PenggunaPage> {
  late Future<List<Pengguna>> _future;

  @override
  void initState() {
    super.initState();
    _future = ambilPengguna();
  }

  void _muatUlang() {
    setState(() {
      _future = ambilPengguna();
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Daftar Pengguna'),
        actions: [
          IconButton(icon: const Icon(Icons.refresh),
              onPressed: _muatUlang),
        ],
      ),
      body: FutureBuilder<List<Pengguna>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState ==
              ConnectionState.waiting) {
            return const Center(
                child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(
```

```
              child: Padding(
                padding: const EdgeInsets.all(24),
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    const Icon(Icons.error_outline,
                        size: 48, color: Colors.red),
                    const SizedBox(height: 8),
                    Text('Terjadi kesalahan:\n${snapshot.error}',
                        textAlign: TextAlign.center),
                    const SizedBox(height: 12),
                    ElevatedButton(
                      onPressed: _muatUlang,
                      child: const Text('Coba lagi'),
                    ),
                  ],
                ),
              ),
            );
          }
          final data = snapshot.data!;
          return ListView.builder(
            itemCount: data.length,
            itemBuilder: (context, i) {
              final p = data[i];
              return ListTile(
                leading: CircleAvatar(child: Text(p.name[0])),
                title: Text(p.name),
                subtitle: Text(p.email),
                trailing: const Icon(Icons.chevron_right),
                onTap: () {
                  Navigator.push(
                    context,
                    MaterialPageRoute(
                      builder: (_) =>
                          DetailPenggunaPage(pengguna: p),
                    ),
                  );
                },
              );
            },
          );
        },
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
_future = ambilPengguna()	Memulai pengambilan data satu kali di initState().

Bagian Kode	Penjelasan
_muatUlang()	Membuat Future baru di dalam setState() sehingga FutureBuilder memantau Future baru dan memuat data lagi. Dipakai oleh tombol refresh dan tombol Coba lagi.
actions: [IconButton(Icons.refresh)]	Tombol refresh di kanan AppBar.
Status loading	Saat connectionState == waiting ditampilkan CircularProgressIndicator di tengah layar.
Status galat	Saat snapshot.hasError ditampilkan ikon merah, pesan galat, dan tombol Coba lagi.
mainAxisSize: MainAxisSize.min	Membuat Column hanya setinggi isinya sehingga tetap rapi di tengah layar.
textAlign: TextAlign.center	Meratakan teks pesan galat ke tengah.
Status data	Bila berhasil, snapshot.data! ditampilkan dengan ListView.builder.
CircleAvatar(child: Text(p.name[0]))	Avatar berisi huruf pertama nama pengguna.
onTap + Navigator.push	Membuka DetailPenggunaPage sambil mengirim objek pengguna yang diketuk.

Bagian E. Halaman Detail

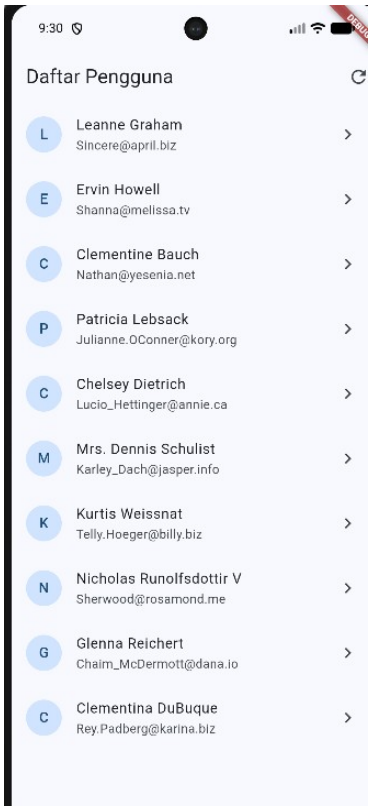
```
class DetailPenggunaPage extends StatelessWidget {
  final Pengguna pengguna;
  const DetailPenggunaPage({super.key, required this.pengguna});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(pengguna.name)),
      body: ListView(
        children: [
          ListTile(
            leading: const Icon(Icons.email),
            title: Text(pengguna.email),
          ),
          ListTile(
            leading: const Icon(Icons.phone),
            title: Text(pengguna.phone),
          ),
          ListTile(
            leading: const Icon(Icons.language),
            title: Text(pengguna.website),
          ),
        ],
      ),
    );
```

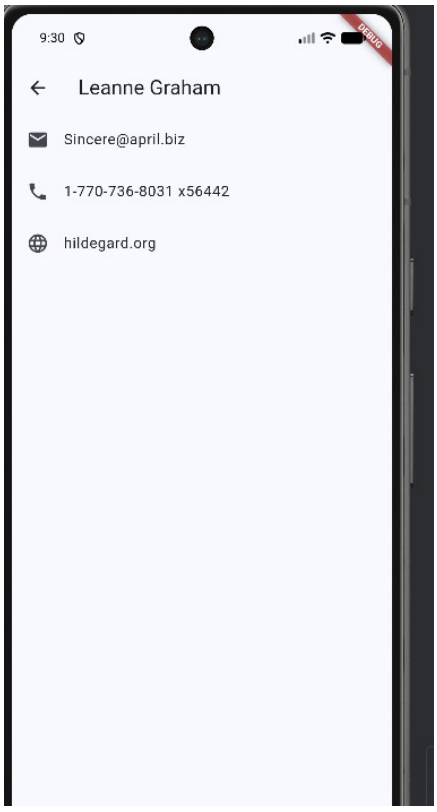
```
  }
}
```

Bagian Kode	Penjelasan
DetailPenggunaPage (StatelessWidget)	Halaman detail yang tidak mengambil data lagi, cukup menampilkan objek yang diterima.
final Pengguna pengguna	Properti penampung data kiriman dari halaman daftar.
required this.pengguna	Membuat parameter wajib diisi lewat constructor.
ListTile(leading: Icon(...), title: Text(...))	Satu baris informasi dengan ikon di kiri (email, telepon, situs web).

Hasil Praktikum Bagian D–E



Gambar 1. Daftar Pengguna dari API (Bagian D)



Gambar 2. Detail pengguna (Bagian E)

Gambar 1 menampilkan 10 pengguna hasil pengambilan data dari JSONPlaceholder setelah indikator loading singkat, lengkap dengan tombol refresh di AppBar. Gambar 2 menampilkan halaman detail pengguna "Leanne Graham" berisi email, telepon, dan situs web.

Bagian F. Percobaan Penanganan Galat

Dua percobaan dilakukan untuk mengamati tampilan galat dan fungsi tombol Coba lagi.

Percobaan	Langkah	Hasil yang Diamati
1. URL salah (404)	Mengubah URL menjadi /userz, menjalankan aplikasi, lalu mengembalikan URL yang benar dan menekan Coba lagi.	Tampil ikon galat merah dengan pesan "Gagal memuat data (kode 404)" dan tombol Coba lagi. Setelah URL dibetulkan dan tombol ditekan, data tampil normal.
2. Tanpa internet	Mematikan koneksi internet emulator, menekan refresh, lalu menyalakan internet dan menekan Coba lagi.	Tampil pesan galat jaringan (misalnya Failed host lookup / SocketException) dan tombol Coba lagi. Setelah internet menyala dan tombol ditekan, data berhasil dimuat.

Percobaan pertama menunjukkan galat dari sisi server (statusCode bukan 200) yang ditangkap lewat throw Exception di ambilPengguna(). Percobaan kedua menunjukkan galat dari sisi jaringan yang dilempar langsung oleh http.get(). Keduanya akhirnya tampil melalui snapshot.error pada FutureBuilder.

5. Latihan Mandiri

Latihan mandiri meliputi: (1) menambah kolom username dan kota (data bersarang json['address']['city']) pada model dan halaman detail, (2) membungkus ListView dengan RefreshIndicator, (3) menampilkan teks "Tidak ada data" bila daftar kosong, dan (4) menampilkan jumlah pengguna di AppBar setelah data dimuat.

Model dengan username dan kota

```
class Pengguna {
  final int id;
  final String name;
  final String username;   // baru
  final String email;
  final String phone;
  final String website;
  final String kota;       // baru (data bersarang)

  const Pengguna({
    required this.id,
    required this.name,
    required this.username,
    required this.email,
    required this.phone,
    required this.website,
    required this.kota,
  });

  factory Pengguna.fromJson(Map<String, dynamic> json) {
    return Pengguna(
```

```
      id: json['id'] as int,
      name: json['name'] as String,
      username: json['username'] as String,
      email: json['email'] as String,
      phone: json['phone'] as String,
      website: json['website'] as String,
      kota: json['address']['city'] as String,
    );
  }
}
```

Jumlah pengguna di AppBar, daftar kosong, dan RefreshIndicator

```
// AppBar menampilkan jumlah pengguna setelah data dimuat
appBar: AppBar(
  title: FutureBuilder<List<Pengguna>>(
    future: _future,
    builder: (context, snapshot) {
      if (snapshot.hasData) {
        return Text(
            'Daftar Pengguna (${snapshot.data!.length})');
      }
      return const Text('Daftar Pengguna');
    },
  ),
  actions: [ /* tombol refresh */ ],
),

// di dalam FutureBuilder, setelah blok loading dan error
final data = snapshot.data!;
if (data.isEmpty) {
  return const Center(child: Text('Tidak ada data'));
}
return RefreshIndicator(
  onRefresh: () async {
    setState(() {
      _future = ambilPengguna();
    });
    await _future;
  },
  child: ListView.builder( /* ... seperti sebelumnya ... */ ),
);
```

Halaman detail dengan username dan kota

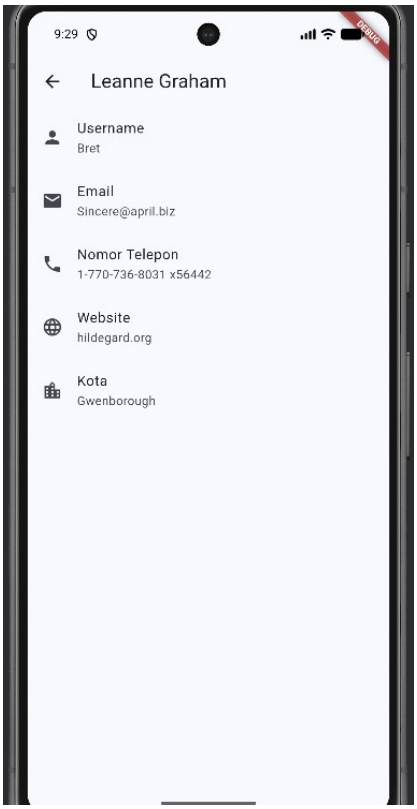
```
// di dalam ListView pada DetailPenggunaPage
ListTile(
  leading: const Icon(Icons.person),
  title: const Text('Username'),
  subtitle: Text(pengguna.username),
),
ListTile(
```

```
  leading: const Icon(Icons.email),
  title: const Text('Email'),
  subtitle: Text(pengguna.email),
),
ListTile(
  leading: const Icon(Icons.phone),
  title: const Text('Nomor Telepon'),
  subtitle: Text(pengguna.phone),
),
ListTile(
  leading: const Icon(Icons.language),
  title: const Text('Website'),
  subtitle: Text(pengguna.website),
),
ListTile(
  leading: const Icon(Icons.location_city),
  title: const Text('Kota'),
  subtitle: Text(pengguna.kota),
),
```

Bagian Kode	Penjelasan
json['address']['city']	Mengakses data bersarang: objek "address" di dalam JSON memiliki kolom "city". Dua kunci ditulis berurutan.
username dan kota	Properti baru pada model Pengguna, juga ditambahkan ke constructor dan fromJson.
title: FutureBuilder<...>	FutureBuilder juga dapat ditaruh di AppBar. Selama belum ada data judul "Daftar Pengguna", setelah data tiba menjadi "Daftar Pengguna (10)".
snapshot.hasData	Bernilai true bila Future sudah selesai dan membawa data.
data.isEmpty	Memeriksa apakah daftar kosong; bila ya tampil teks "Tidak ada data".
RefreshIndicator	Membungkus ListView agar dapat ditarik ke bawah (pull-to-refresh) untuk memuat ulang data.
onRefresh: () async { ... await _future; }	Fungsi yang dijalankan saat ditarik. Harus mengembalikan Future; await _future membuat animasi refresh berhenti setelah data selesai dimuat.
ListTile(title, subtitle)	Detail kini memakai title sebagai label dan subtitle sebagai nilai, seperti pada Gambar 4.
Icons.location_city	Ikon gedung untuk baris Kota.



Gambar 3. AppBar menampilkan jumlah pengguna "(10)"



Gambar 4. Detail dengan username dan kota

Pada Gambar 3 judul AppBar berubah menjadi "Daftar Pengguna (10)" setelah data berhasil dimuat. Pada Gambar 4 halaman detail "Leanne Graham" kini menampilkan Username (Bret), Email, Nomor Telepon, Website, dan Kota (Gwenborough) yang diambil dari data bersarang address.city.

6. Tugas: Aplikasi Daftar Postingan

Aplikasi Daftar Postingan memakai endpoint /posts dan /posts/{id}/comments. Halaman utama menampilkan daftar postingan (judul dan potongan isi) dengan FutureBuilder. Mengetuk postingan membuka halaman detail berisi isi lengkap dan daftar komentar yang diambil dari endpoint terpisah (Future kedua). Model Post dan Komentar memakai fromJson, kode pengambil data dipisah dari kode UI, dan kedua halaman menangani status loading serta galat lengkap dengan tombol coba lagi.

Model

```
// ===== MODEL =====
class Post {
  final int id;
  final String title;
  final String body;
```

```
  const Post({required this.id, required this.title,
      required this.body});

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'] as int,
      title: json['title'] as String,
      body: json['body'] as String,
    );
  }
}

class Komentar {
  final int id;
  final String name;
  final String email;
  final String body;

  const Komentar({required this.id, required this.name,
      required this.email, required this.body});

  factory Komentar.fromJson(Map<String, dynamic> json) {
    return Komentar(
      id: json['id'] as int,
      name: json['name'] as String,
      email: json['email'] as String,
      body: json['body'] as String,
    );
  }
}
```

Service pengambil data (terpisah dari UI)

```
// ===== SERVICE (terpisah dari UI) =====
class PostService {
  static const _baseUrl =
      'https://jsonplaceholder.typicode.com';

  static Future<List<Post>> ambilPosts() async {
    final response = await http
        .get(Uri.parse('$_baseUrl/posts'))
        .timeout(const Duration(seconds: 10));
    if (response.statusCode != 200) {
      throw Exception(
          'Gagal memuat postingan (kode ${response.statusCode})');
    }
    final List<dynamic> data = jsonDecode(response.body);
    return data
        .map((e) => Post.fromJson(e as Map<String, dynamic>))
        .toList();
  }

  static Future<List<Komentar>> ambilKomentar(int postId) async {
```

```
    final response = await http
        .get(Uri.parse('$_baseUrl/posts/$postId/comments'))
        .timeout(const Duration(seconds: 10));
    if (response.statusCode != 200) {
      throw Exception(
          'Gagal memuat komentar (kode ${response.statusCode})');
    }
    final List<dynamic> data = jsonDecode(response.body);
    return data
        .map((e) => Komentar.fromJson(e as Map<String, dynamic>))
        .toList();
  }
}
```

Halaman daftar postingan

```
// ===== UI: Daftar Postingan =====
class _PostPageState extends State<PostPage> {
  late Future<List<Post>> _future;

  @override
  void initState() {
    super.initState();
    _future = PostService.ambilPosts();
  }

  void _muatUlang() {
    setState(() => _future = PostService.ambilPosts());
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Daftar Postingan'),
        actions: [
          IconButton(icon: const Icon(Icons.refresh),
              onPressed: _muatUlang),
        ],
      ),
      body: FutureBuilder<List<Post>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState ==
              ConnectionState.waiting) {
            return const Center(
                child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return _TampilanError(
              pesan: '${snapshot.error}',
              onCobaLagi: _muatUlang,
            );
```

```
          }
          final posts = snapshot.data!;
          return ListView.separated(
            itemCount: posts.length,
            separatorBuilder: (_, __) => const Divider(height: 1),
            itemBuilder: (context, i) {
              final p = posts[i];
              return ListTile(
                leading: CircleAvatar(child: Text('${p.id}')),
                title: Text(p.title,
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: const TextStyle(
                        fontWeight: FontWeight.bold)),
                subtitle: Text(p.body,
                    maxLines: 2,
                    overflow: TextOverflow.ellipsis),
                trailing: const Icon(Icons.chevron_right),
                onTap: () => Navigator.push(
                  context,
                  MaterialPageRoute(
                      builder: (_) => DetailPostPage(post: p)),
                ),
              );
            },
          );
        },
      ),
    );
  }
}

// Widget error yang dipakai ulang di kedua halaman
class _TampilanError extends StatelessWidget {
  final String pesan;
  final VoidCallback onCobaLagi;
  const _TampilanError(
      {required this.pesan, required this.onCobaLagi});

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Icon(Icons.error_outline,
                size: 48, color: Colors.red),
            const SizedBox(height: 8),
            Text('Terjadi kesalahan:\n$pesan',
                textAlign: TextAlign.center),
            const SizedBox(height: 12),
            ElevatedButton(onPressed: onCobaLagi,
```

```
                child: const Text('Coba lagi')),
          ],
        ),
      ),
    );
  }
}
```

Halaman detail postingan dan komentar

```
// ===== UI: Detail Postingan + Komentar (Future kedua) =====
class _DetailPostPageState extends State<DetailPostPage> {
  late Future<List<Komentar>> _futureKomentar;

  @override
  void initState() {
    super.initState();
    _futureKomentar =
        PostService.ambilKomentar(widget.post.id);
  }

  void _muatUlang() {
    setState(() => _futureKomentar =
        PostService.ambilKomentar(widget.post.id));
  }

  @override
  Widget build(BuildContext context) {
    final post = widget.post;
    return Scaffold(
      appBar: AppBar(title: const Text('Detail Postingan')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text(post.title,
              style: const TextStyle(
                  fontSize: 22, fontWeight: FontWeight.bold)),
          const SizedBox(height: 12),
          Text(post.body, style: const TextStyle(fontSize: 16)),
          const Divider(height: 32),
          const Text('Komentar',
              style: TextStyle(fontWeight: FontWeight.bold)),
          const SizedBox(height: 8),
          FutureBuilder<List<Komentar>>(
            future: _futureKomentar,
            builder: (context, snapshot) {
              if (snapshot.connectionState ==
                  ConnectionState.waiting) {
                return const Center(
                    child: Padding(
                        padding: EdgeInsets.all(24),
                        child: CircularProgressIndicator()));
              }
```

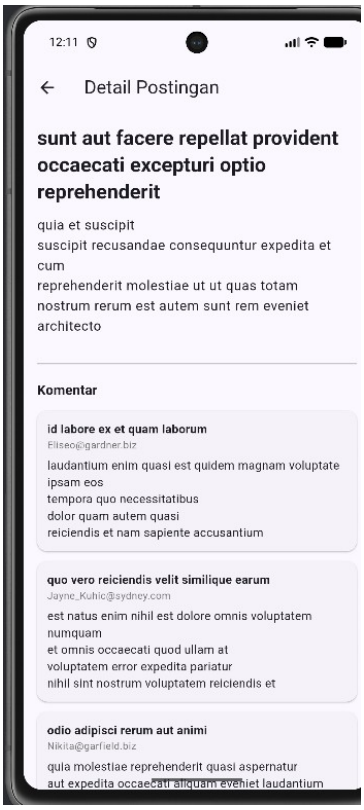
```
              if (snapshot.hasError) {
                return _TampilanError(
                  pesan: '${snapshot.error}',
                  onCobaLagi: _muatUlang,
                );
              }
              return Column(
                children: snapshot.data!
                    .map((k) => Card(
                          child: ListTile(
                            title: Text(k.name,
                                style: const TextStyle(
                                    fontWeight: FontWeight.bold,
                                    fontSize: 14)),
                            subtitle: Column(
                              crossAxisAlignment:
                                  CrossAxisAlignment.start,
                              children: [
                                Text(k.email,
                                    style: const TextStyle(
                                        color: Colors.grey)),
                                const SizedBox(height: 4),
                                Text(k.body),
                              ],
                            ),
                          ),
                        ))
                    .toList(),
              );
            },
          ),
        ],
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
class Post	Model satu postingan: id, title, dan body.
class Komentar	Model satu komentar: id, name, email, dan body.
Post.fromJson / Komentar.fromJson	Mengubah Map JSON menjadi objek model sehingga data bertipe jelas dan aman dipakai di UI.
class PostService	Lapisan layanan yang berisi seluruh kode pengambil data, sehingga UI tidak bercampur dengan urusan HTTP (pemisahan model, layanan, dan UI).
static const _baseUrl	Alamat dasar API yang disimpan sekali agar tidak diulang di tiap fungsi.

Bagian Kode	Penjelasan
static Future<...> ambilPosts()	Method static dapat dipanggil langsung lewat nama kelas (PostService.ambilPosts()) tanpa membuat objek.
ambilKomentar(int postId)	Mengambil komentar dari endpoint /posts/{id}/comments; id postingan dimasukkan ke URL lewat string interpolation.
PostPage + FutureBuilder	Halaman utama yang memantau Future daftar postingan dan menangani tiga status: loading, galat, dan data.
ListView.separated	Seperti ListView.builder, ditambah separatorBuilder untuk menyisipkan garis pemisah (Divider) antar item.
(_, __) => const Divider(height: 1)	Parameter yang tidak dipakai ditulis dengan underscore.
maxLines + TextOverflow.ellipsis	Membatasi jumlah baris dan memotong teks panjang dengan tanda "...", sehingga hanya potongan isi yang tampil di daftar (judul 1 baris, isi 2 baris).
CircleAvatar(child: Text('${p.id}'))	Avatar berisi nomor postingan.
_TampilanError	Widget buatan sendiri untuk tampilan galat dan tombol Coba lagi. Dipakai ulang di dua halaman sehingga kode tidak diduplikasi.
VoidCallback onCobaLagi	Tipe fungsi tanpa parameter dan tanpa nilai kembalian; dikirim dari halaman agar tombol dapat memuat ulang data.
DetailPostPage (StatefulWidget)	Stateful karena memegang Future komentar (_futureKomentar) yang dapat dimuat ulang.
widget.post	Cara State mengakses data milik widget induknya (post yang dikirim lewat constructor).
Future kedua	Isi lengkap postingan sudah ada dari halaman daftar, sedangkan komentar diambil dari endpoint terpisah lewat Future baru di initState().
FutureBuilder di dalam ListView	Bagian komentar punya FutureBuilder sendiri sehingga isi postingan tampil langsung sementara komentar masih loading atau gagal secara terpisah.
Card + ListTile komentar	Setiap komentar tampil dalam kartu berisi nama (tebal), email (abu-abu), dan isi komentar.
.map((k) => Card(...)).toList()	Mengubah daftar Komentar menjadi daftar widget Card.



Gambar 5. Halaman daftar postingan



Gambar 6. Detail postingan dan daftar komentar

Gambar 5 menampilkan daftar postingan bernomor dengan judul tebal satu baris dan potongan isi dua baris. Gambar 6 menampilkan detail postingan nomor 1: judul, isi lengkap, garis pemisah, lalu bagian "Komentar" yang datang dari Future kedua dalam bentuk kartu berisi nama, email, dan isi komentar.

7. Jawaban Pertanyaan Refleksi

1. Apa yang terjadi bila Future dibuat di dalam build() dan bukan di initState()? Mengapa?

Pengambilan data akan diulang setiap kali build() dipanggil. build() dapat dipanggil berkali-kali, misalnya saat setState, layar diputar, atau keyboard muncul. Setiap panggilan membuat Future baru, sehingga FutureBuilder kembali ke status loading, data diminta ulang dari server, dan tampilan berkedip tanpa henti. initState() hanya dijalankan sekali saat State dibuat, jadi Future hanya dibuat satu kali dan hasilnya dipertahankan.

2. Apa perbedaan snapshot.hasError dengan memeriksa response.statusCode? Mengapa keduanya diperlukan?

response.statusCode memeriksa apakah server berhasil membalas dengan benar. Server bisa saja membalas, tetapi dengan kode gagal seperti 404 atau 500, dan itu tidak melempar galat dengan sendirinya. Karena itu kita memeriksa statusCode lalu melempar Exception sendiri.

snapshot.hasError adalah sisi UI: ia menangkap semua Future yang gagal, baik Exception buatan kita maupun galat jaringan (misalnya tidak ada internet atau timeout) yang dilempar langsung oleh http.get(). Keduanya diperlukan karena statusCode hanya mengenali kegagalan dari server, sedangkan hasError memastikan semua jenis kegagalan sampai ke layar sebagai pesan galat.

3. Mengapa data JSON sebaiknya diubah menjadi class model, bukan dipakai langsung sebagai Map?

Class model memberi tipe data yang jelas dan aman. Compiler dapat memeriksa kesalahan penulisan nama properti, editor memberi saran otomatis (autocomplete), dan konversi tipe dilakukan sekali di fromJson. Bila memakai Map langsung, nama kunci berupa String yang mudah salah ketik, tipe nilainya dynamic sehingga kesalahan baru muncul saat aplikasi berjalan, dan kode menjadi berantakan karena tersebar di banyak tempat. Model juga memudahkan perawatan karena perubahan struktur data cukup diperbaiki di satu class.

4. Mengapa UI wajib menyediakan status loading dan error, bukan hanya status data?

Pengambilan data dari internet tidak instan dan tidak selalu berhasil. Tanpa status loading, layar tampak kosong atau macet sehingga pengguna mengira aplikasi rusak. Tanpa status error, ketika internet mati atau server bermasalah aplikasi bisa crash atau menampilkan layar kosong tanpa penjelasan. Dengan tiga status (loading, error, data) dan tombol Coba lagi, pengguna selalu tahu apa yang sedang terjadi dan bisa mencoba kembali.

8. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa pengambilan data dari internet bersifat asinkron sehingga memakai Future, async, dan await agar UI tidak membeku. Paket http dipakai untuk mengirim permintaan GET, jsonDecode untuk menguraikan JSON, dan class model dengan fromJson untuk mengubahnya menjadi objek bertipe. FutureBuilder menampilkan tiga status secara rapi: loading, galat, dan data. Future harus dibuat di initState() agar tidak berulang, dan kegagalan jaringan ditangani dengan pesan galat serta tombol Coba lagi. Pada tugas, pemisahan model, layanan (PostService), dan UI membuat kode lebih rapi, dan Future kedua dipakai untuk memuat komentar secara terpisah dari isi postingan.